

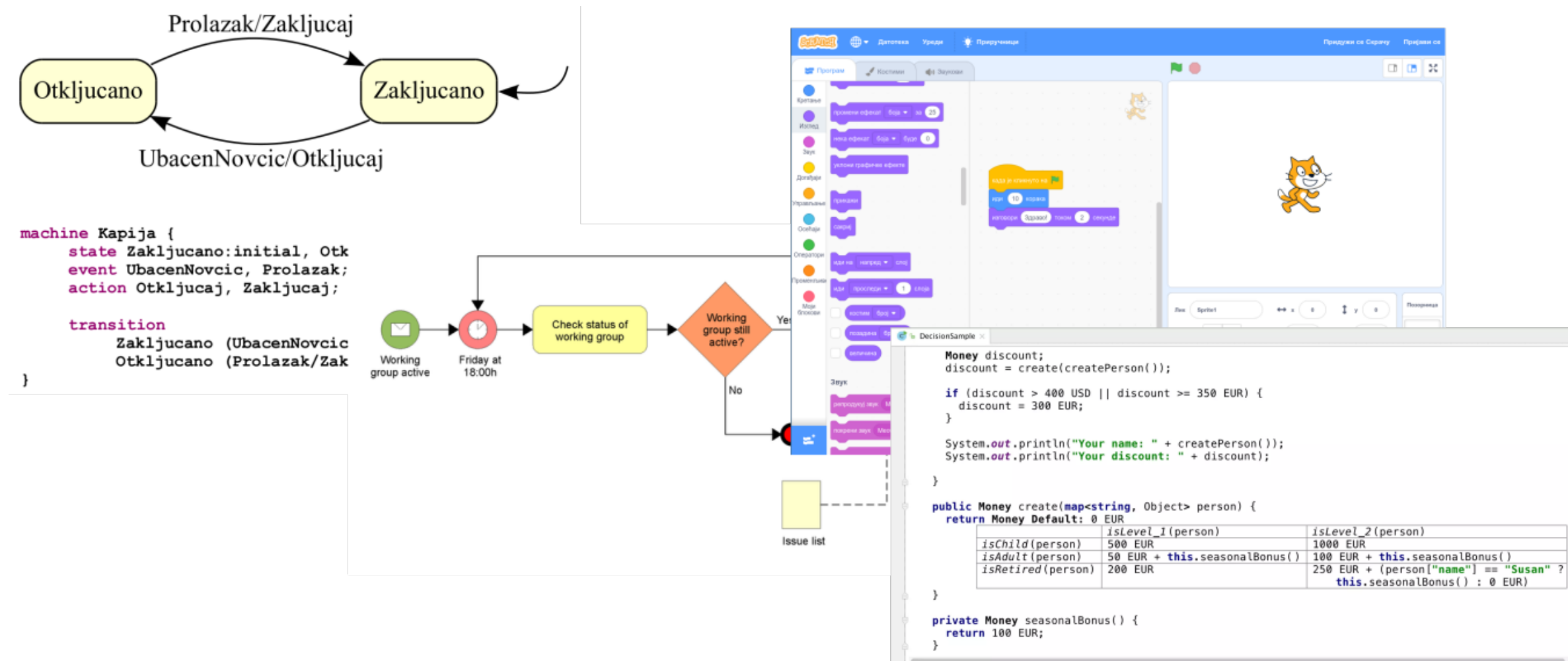
Developing a Domain-Specific Language for Cognitive Tests Using TextX

Igor Dejanović and Mirjana Dejanović



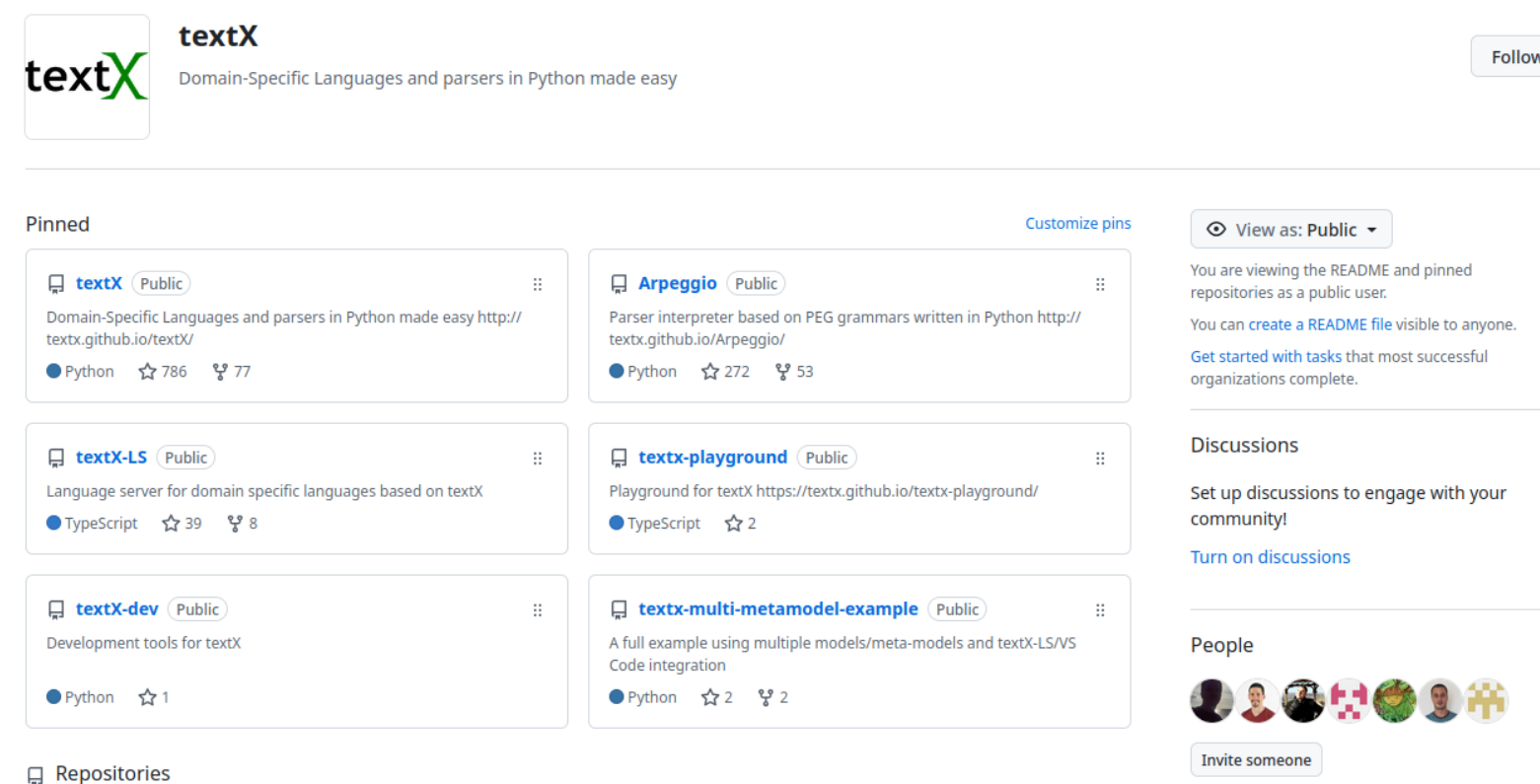
Domain-Specific Languages

- Specialized languages designed for domain experts
- Appropriate level of abstraction
- Domain concepts
- Boost in productivity – 10x and more
- Today popular sub-class: Low Code / No Code
- But not easy to develop and maintain

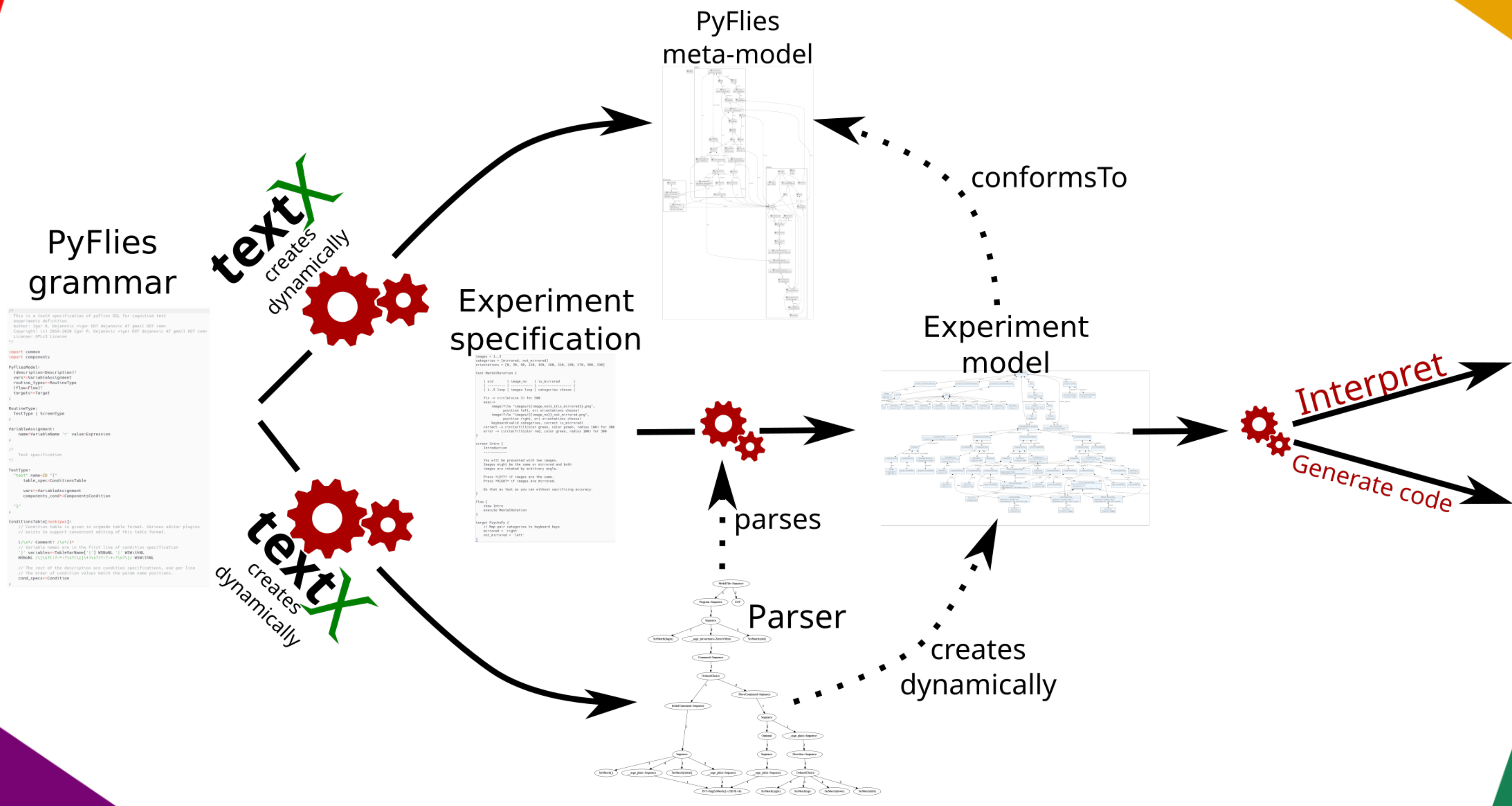


textX to the rescue

- DSLs in Python made easy
- We started the project in 2014.
- Open-source - MIT License
- 786 stars, 77 forks and 24 contributors on GitHub as of this writing.
- Used in academia and industry for DSL development, data extraction, IDE language support, legacy code analysis etc.



textX - workflow



For what is textX used?

- textX is used to implement various DSLs, including:
 - Software construction
 - Source code analysis and representation
 - e-health
 - LLM enabled domain modeling
 - Microservice architectures
 - Mobile app development
 - Game development
 - Power electronics models specification
- In this work, we showcase its use in the specification of cognitive tests.

Cognitive tests

- Used to assess the cognitive capabilities of human or animal subjects.
- Physical changes can be evaluated using CT scans, MRI, etc., but how do we assess cognitive changes?
- GUI designers vs. general-purpose programming languages.
- We developed PyFlies, a DSL for cognitive tests, to bridge the gap.
- Test specification by domain experts, such as psychologists and neurophysiologists.
- Our focus is primarily on reaction time tests.

PyFlies

Domain analysis

- Talk with domain experts
- Take notes
- Identify nouns (concepts) and verbs (actions/procedures)
- Examples: `Test`, `Trial`, `Stimulus`, `Flow`, etc.

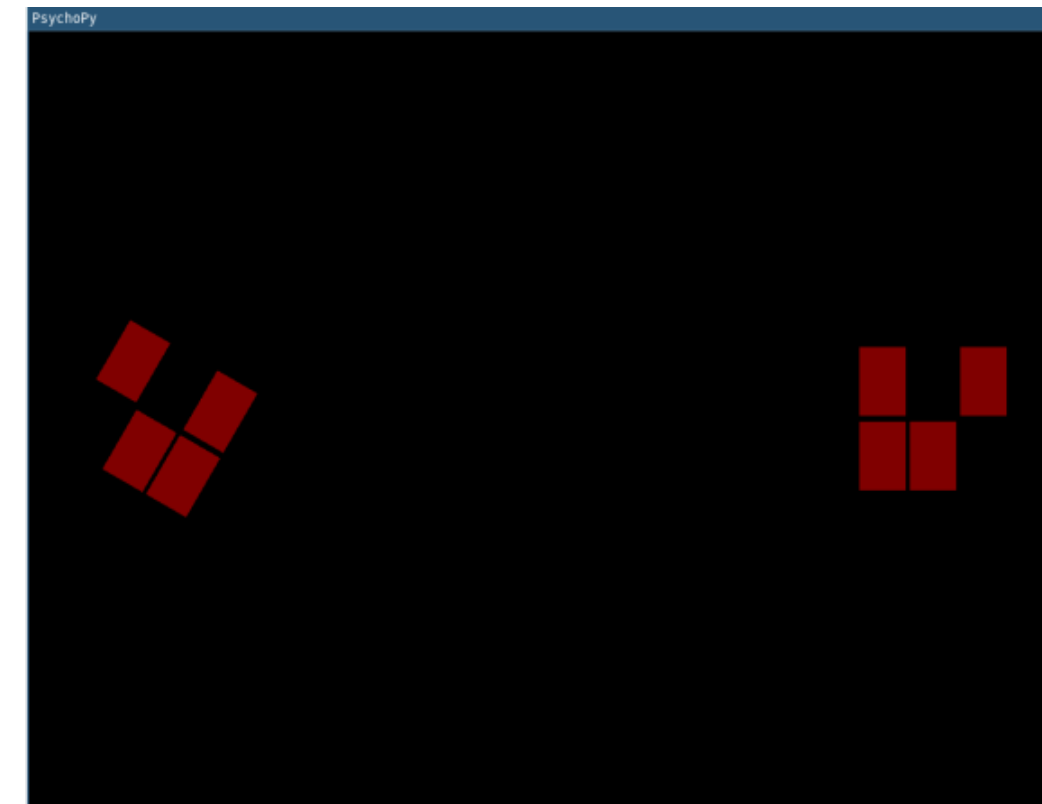
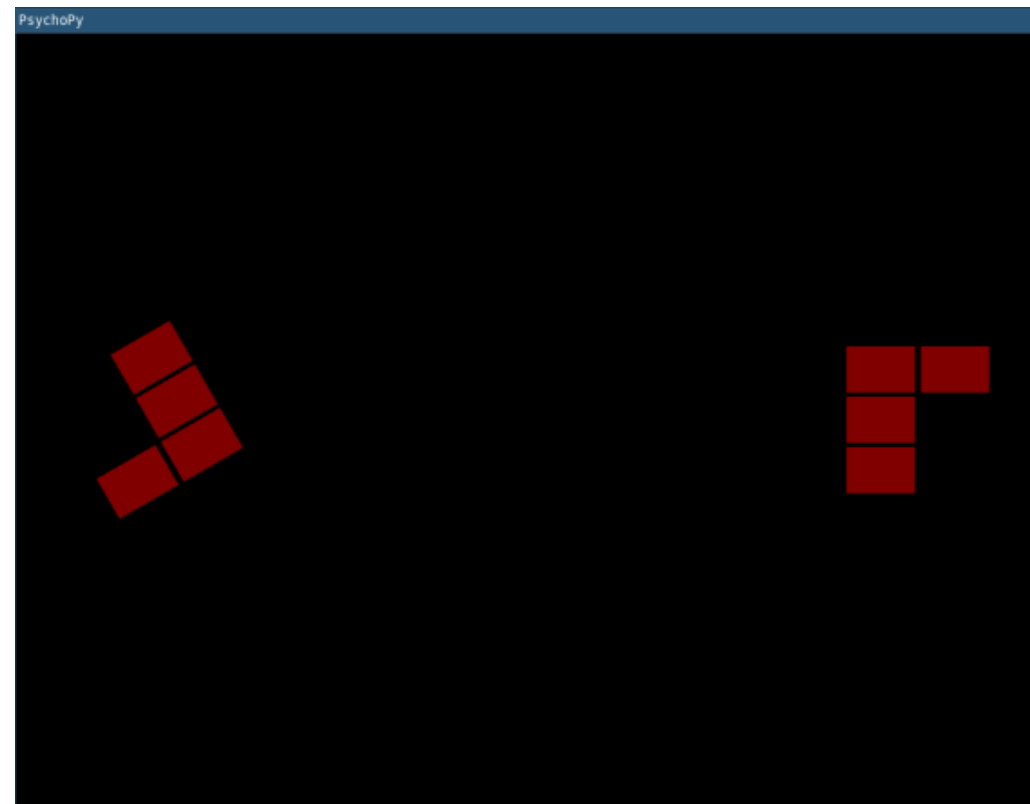
Design the syntax and meta-model in textX

```
TestType:  
  "test" name=ID "{"  
    table_spec=ConditionsTable  
  
    vars*=VariableAssignment  
    components_cond*=ComponentsCondition  
  
  "}"  
;
```

Mental Rotation Test

Mental Rotation Test

- The subject is presented with two images rotated relative to each other by a random degree
- The images are either identical or mirrored.
- Task: determine as quickly as possible whether the images are mirrored or not.



Mental Rotation Test implementation

```
1  images = 1..3
2  categories = [mirrored, not_mirrored]
3  orientations = [0, 30, 90, 120, 150, 180, 210, 240, 270, 300, 330]
4
5  √ test MentalRotation {
6
7      | ord      | image_no  | is_mirrored |
8      | ----- | ----- | ----- |
9      | 1..5 loop | images loop | categories choose |
10
11     fix -> circle(size 3) for 500
12  √ exec->
13  √     image(file "images/{{image_no}}_{{is_mirrored}}.png",
14         |      | position left, ori orientations choose)
15  √     image(file "images/{{image_no}}_not_mirrored.png",
16         |      | position right, ori orientations choose)
17     keyboard(valid categories, correct is_mirrored)
18     correct -> circle(fillColor green, color green, radius 100) for 300
19     error -> circle(fillColor red, color green, radius 100) for 300
20 }
```


The rest of the experiment

```
22 screen Intro {
23     Introduction
24     -----
25
26     You will be presented with two images.
27     Images might be the same or mirrored and both
28     images are rotated by arbitrary angle.
29
30     Press *LEFT* if images are the same.
31     Press *RIGHT* if images are mirrored.
32
33     Do that as fast as you can without sacrificing accuracy.
34 }
35
36 flow {
37     show Intro
38     execute MentalRotation
39 }
40
41 target PsychoPy {
42     // Map pair categories to keyboard keys
43     mirrored = 'right'
44     not_mirrored = 'left'
45 }
```

Generated Python code

```

1 images = 1..3
2 categories = [mirrored, not_mirrored]
3 orientations = [0, 30, 90, 120, 150, 180, 210, 240, 270, 300, 330]
4
5 test MentalRotation {
6
7     | ord      | image_no | is_mirrored |
8     | ----- | - - - - - | - - - - - |
9     | 1..5 loop | images loop | categories choose |
10
11     fix -> circle(size 3) for 500
12     exec->
13         image(file "images/{{image_no}}_{{is_mirrored}}.png",
14             position left, ori orientations choose)
15         image(file "images/{{image_no}}_not_mirrored.png",
16             position right, ori orientations choose)
17         keyboard(valid categories, correct is_mirrored)
18     correct -> circle(fillColor green, color green, radius 100) for 300
19     error -> circle(fillColor red, color green, radius 100) for 300
20 }
21
22 screen Intro {
23     Introduction
24     -----
25
26     You will be presented with two images.
27     Images might be the same or mirrored and both
28     images are rotated by arbitrary angle.
29
30     Press *LEFT* if images are the same.
31     Press *RIGHT* if images are mirrored.
32
33     Do that as fast as you can without sacrificing accuracy.
34 }
35
36 flow {
37     show Intro
38     execute MentalRotation
39 }
40
41 target PsychoPy {
42     // Map pair categories to keyboard keys
43     mirrored = 'right'
44     not_mirrored = 'left'
45 }

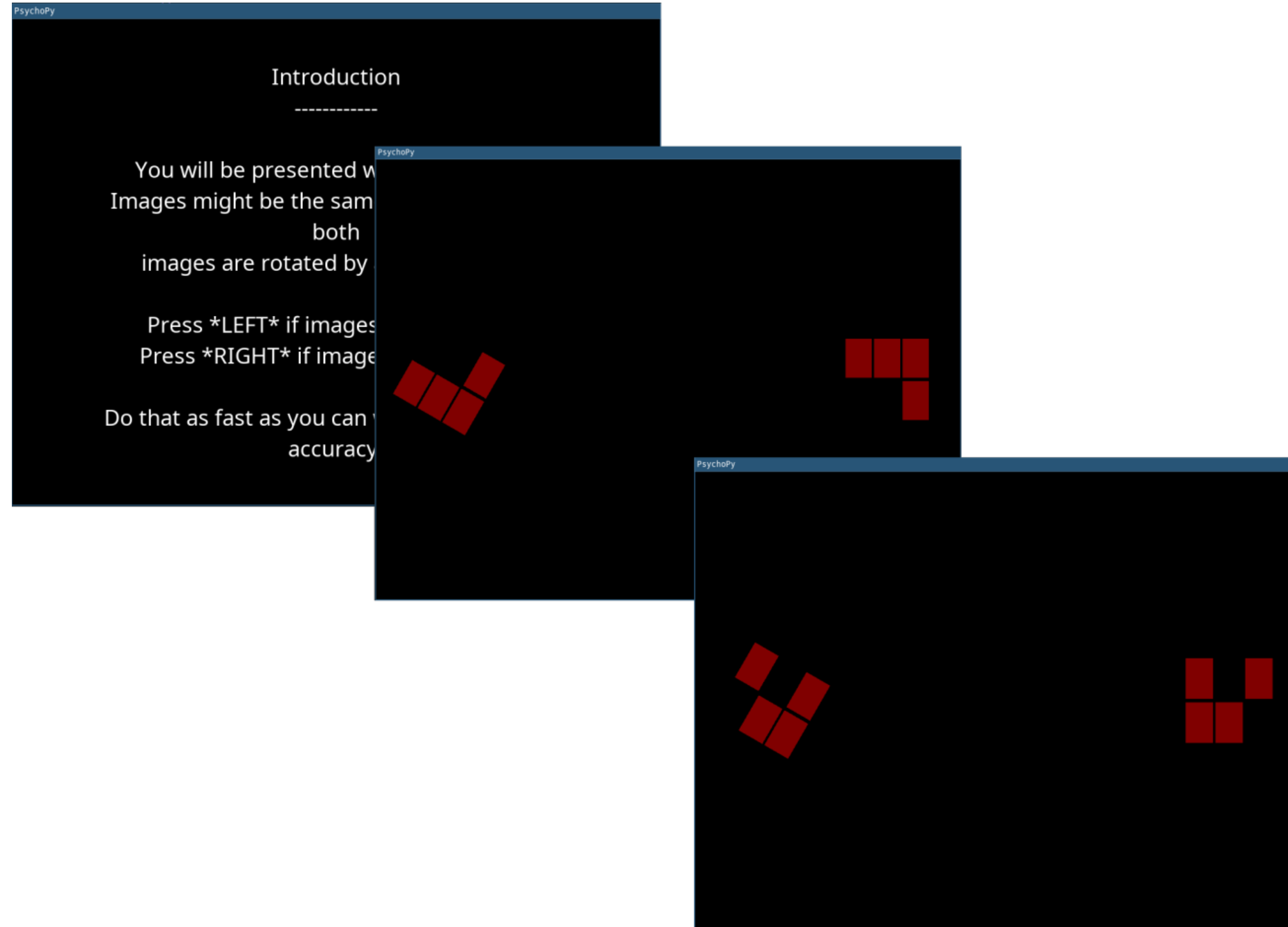
```

```

797 def execute_test(test_trials, random=False, practice=False):
798     """
799     Execute given test. Run components with the given timing and collect
800     responses.
801     """
802
803     def execute_phase(components, variables=None, should_record=False):
804         if not components:
805             return
806         # reset timers
807         t = 0
808         _timeToFirstFrame = win.getFutureFlipTime(clock="now")
809         trialClock.reset(-_timeToFirstFrame) # t0 is time of first possible flip
810         frameN = -1
811         continueTest = True
812         correct_response = True
813
814         while continueTest:
815             t = trialClock.getTime()
816             tThisFlip = win.getFutureFlipTime(clock=trialClock)
817             tThisFlipGlobal = win.getFutureFlipTime(clock=None)
818             frameN = frameN + 1 # number of completed frames (so 0 is the first frame)
819
820             # update/draw components on each frame
821             for component in components:
822                 cinst = component['inst']
823                 ctype = component['type']
824
825                 if cinst.status == NOT_STARTED \
826                     and tThisFlip >= component['at'] - frameTolerance:
827                     # keep track of start time/frame for later
828                     cinst.frameNStart = frameN # exact frame index
829                     cinst.tStart = t # local t and not account for scr refresh
830                     cinst.tStartRefresh = tThisFlipGlobal # on global time
831                     win.timeOnFlip(cinst, 'tStartRefresh') # time at next scr refresh
832                     cinst.status = STARTED
833
834                     # Set component parameters for this trial
835                     for pname, pval in component['params'].items():
836                         setattr(cinst, pname, pval)
837
838                     # If component is audible start playing
839                     if 'audible' in ctype:
840                         cinst.play()
841                     elif 'mouse' in ctype:
842                         cinst.mouseClock.reset()
843                         # if button is down already this ISN'T a new click
844                         prevButtonState = cinst.getPressed()
845                     elif 'keyboard' in ctype:
846                         cinst.getKeys()
847                         cinst.clock.reset()
848                         cinst.start()
849
850                     # Display component on the next flip
851                     if 'visual' in ctype:
852                         cinst.setAutoDraw(True)
853
854             if 'keyboard' in ctype and cinst.status == STARTED:
855                 keys = cinst.getKeys(cinst.valid_keys, waitRelease=False)
856                 if 'quit' in keys:
857                     core.quit()

```


Running the test



Results

Table 1: Lines of Code (LOC) Metrics for the Implementation of PyFlies

File type	LOC
Python	993
textX grammar	175
Jinja template	80

- Language has 34 concepts
- From 175 grammar lines textX dynamically produces a meta-model and a full parser
- Jinja template lines are for built-in generators - CSV and Log.

Table 2: Lines of Code (LOC) Metrics for the Implementation of Mental Rotation Test

File type	LOC
PyFlies	35
Python	873

- From 35 lines of high-level PyFlies code we generate 873 lines of Python code.
- ~25x reduction in code size -> huge increase in productivity

Conclusion

- textX is a powerful tool for enhancing productivity through the use of Domain-Specific Languages (DSLs).
- It empowers non-programmer domain experts to design and implement tests effectively.
- All resources presented here are free and open-source:
 - [textX project](#) and [documentation](#)
 - [textX playground](#)
 - [PyFlies project](#), [documentation](#) and [overview video](#)
- Further Work
 - Ongoing evaluation of textX for large-scale industrial DSLs.



Thanks!

Q&A